

The paper asks how a learned internal model can help an agent act. We will examine two uses: predictive features for a controller, and a learned simulator for training it. The main design separates vision, prediction, and control, so reward optimisation can focus on a small controller. Here is the route through those ideas.

└ Outline

We start with the world, the agent, and their interaction loop. Then we define a world model and examine V, M, and C. The two experiments show how this architecture supports control in the actual game and training inside a dream. We finish by comparing its design with recent models. First, which part chooses actions, and which part responds?

Ha & Schmidhuber (2018) - World Models

└ World, Agent, and World Model

└ World and Agent

World

The system whose changes matter for the task: its entities, their current state, and how they evolve under actions. In robotics, this includes the robot's body and its surroundings.

Agent

The decision-making system that receives observations and chooses actions to pursue a goal. Its rule for choosing actions is called a **policy**.

The world is the task-relevant system we want to understand and influence. For a robot pushing a cup, it includes the cup, table, and robot's physical body. The agent makes decisions; its policy selects actions from the information available. An agent can also contain perception, memory, and prediction. Here the controller chooses commands, while the robot's body and surroundings respond. Let us follow one interaction between them.

Ha & Schmidhuber (2018) - World Models

└ World, Agent, and World Model

└ The Agent–World Loop



- Observation o_t is what the agent senses about the world state s_t .
- In RL, reward r_{t+1} gives feedback about the task after action a_t .

The agent observes o_t and uses policy π to choose action a_t . The upper arrow sends that action to the world, whose state changes to s_{t+1} . The lower arrow returns observation o_{t+1} and task feedback r_{t+1} , and the loop repeats. Observation tells us what is sensed; reward tells us how the task scores the interaction. Neither gives the complete physical state. For example, similar images can hide different cup velocities, so earlier observations and actions can matter. A POMDP describes this same loop mathematically, which is the next step.

Ha & Schmidhuber (2018) - World Models

└ World, Agent, and World Model

└ The Agent–World Loop as a POMDP

The agent–world loop can be described as a partially observable Markov decision process (POMDP):

States s_t	The world's physical situation, including the robot's body
Actions a_t	Commands chosen by the agent
Transition $T(s_{t+1} s_t, a_t)$	Distribution of the next state after an action
Observation $O(o_t s_t)$	Distribution of what the agent senses in a state
Reward $R(s_t, a_t)$	Expected immediate task feedback

The agent receives observations rather than direct access to the full state. The state evolves under action; the agent senses it through observations.

This table gives mathematical names to the parts of the interaction loop. State describes the underlying situation; transition describes how an action changes it. Observation describes what the agent senses, and reward gives immediate task feedback. Partial observability means the agent receives incomplete information, such as an image without contact forces or velocity. History can help infer what is hidden. The POMDP describes the problem; the agent may still need to learn its dynamics from data. Here the observation function depends only on state; general formulations can also include the preceding action. That leads to a learned world model.

Ha & Schmidhuber (2018) - World Models

└ World, Agent, and World Model

└ What a World Model is

World model

Given the history of observations and actions

$$H_t = (o_0, a_0, o_1, a_1, \dots, o_{t-1}, a_{t-1})$$

and a candidate action a_t , a **world model** predicts a probability distribution over the next observation o_{t+1} :

$$p_\theta(o_{t+1} | H_t, a_t)$$

where θ denotes the model's learned parameters.• **Policy**: "What should I do?"• **World model**: "What would happen if I did this?"

H_t contains observations through o_t and the actions already taken. The candidate a_t is separate because we are asking what would happen if we chose it now. The model predicts a distribution over the next observation, using parameters θ learned from interactions. A distribution can represent several possible outcomes. For the cup, pushing and doing nothing should produce different predictions. The policy chooses an action; the world model predicts its consequences. Prediction alone does not say which outcome is desirable, so next we distinguish dynamics, value, and policy.

Ha & Schmidhuber (2018) - World Models

- World, Agent, and World Model

- What a World Model is *NOT*

Component	Predicts	Policy-dependent?	Belongs to
Dynamics $p(s_{t+1} s_t, a_t)$	next state	no	world model
Reward model $r(s_t, a_t)$	immediate reward	no	world model (optional)
Value $V^\pi(s) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_{t+1}]$	future return	yes	agent (critic)
Policy $\pi(a_t s_t)$	action	—	agent (actor)

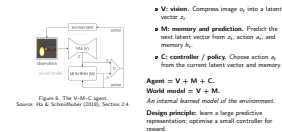
Dynamics and reward describe immediate consequences of a specified state and action. $V^\pi(s)$ estimates discounted return from state s when following policy π . Changing the policy can change the visited states and rewards, so the same state can have a different value. A policy that reaches the target can earn more than one that moves away, even though the environment is unchanged. By contrast, $T(s' | s, a)$ fixes both state and action, so the physical transition rule does not depend on the policy. A value network learns policy-associated targets; it need not receive π explicitly. $V^*(s)$ instead maximises over policies. These are functional roles within an agent; the world model can itself be an agent component. We now examine V, M, and C.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ Architecture Overview

└ The V-M-C Architecture



In this diagram, the world is the actual game environment. For Car Racing, it includes the car, track, and movement rules; for VizDoom, the player, fireballs, walls, and game rules. The controlled car or player is part of the game state. The agent is the V-M-C software: it receives images and sends control commands to the game. V turns an image into z_t , a short vector of learned numbers that summarises it. M remembers earlier information and predicts how this latent vector changes after an action. C uses the current latent vector and memory to choose that action. Together, V, M, and C form the agent; V plus M form its internal world model. The world model represents the game, but is not the actual game or its full state. During dream training, M supplies simulated transitions to C, so model errors can affect transfer. We start with how V summarises an image.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ Vision (V)

└ V: Vision with a Convolutional VAE

**Architecture**

- Encoder: four convolutional layers compress a 64×64 RGB image.
- Sample from the encoder's Gaussian:

$$z_t = \mu_t + \sigma_t \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$
- Decoder: four transposed convolutions.

Training: reconstruction + KL; no reward.**Design choices**

- Compact latent vector: 32 values (Car Racing), 64 (VisDoom).
- Gaussian prior: KL encourages encoder distributions towards $\mathcal{N}(0, I)$.

V makes a small learned summary of the current image: the latent vector z_t . Its entries are learned features, not hand-labelled quantities such as position or speed. VAE means variational autoencoder: an encoder compresses the image, and a decoder learns to reconstruct it. Four convolutional layers learn image patterns and reduce the 64×64 RGB input. The encoder outputs a mean μ_t and spread σ_t (standard deviation). The equation adds random noise scaled by σ_t to μ_t , entry by entry, to sample z_t . The decoder uses four transposed convolutions to build the image back from z_t . Reconstruction loss rewards keeping visual information. KL divergence measures a difference between distributions; here it encourages the encoder's distribution towards the reference Gaussian $\mathcal{N}(0, I)$. This regularisation is intended to make M's generated latent vectors more usable, without guaranteeing correct physics. The small vector makes M's prediction cheaper than using all pixels. V learns without task reward, and its decoder is optional when C acts. Next, M learns how these summaries change over time.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ Memory (M)

└ M: Memory with an MDN-RNN

M predicts a distribution of possible next latent vectors from the current z_t , action a_t , and memory h_t . An LSTM, or long short-term memory network, updates memory at each step; history can reveal motion hidden in one image. MDN means mixture density network: it combines five Gaussian bell curves for each predicted vector entry. Each curve has a mean, spread, and weight; weights are nonnegative and sum to one. Factored means entries are predicted independently given the same inputs and memory. The training target is V's latent vector for the next recorded image. Negative log-likelihood gives lower loss when M assigns that target higher density:

$$\mathcal{L}_M = -\text{mean}_{t,i} \log p_{\theta,i}(z_{t+1,i}^{\text{data}} \mid z_t, a_t, h_t).$$

The mean averages over steps and entries; the minus sign turns likelihood maximisation into loss minimisation. Continuous values use density, not probability at one exact point. Gradients train M without task reward; VizDoom also learns when an episode ends. Next is C.

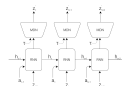


Figure 6. MDN-RNN unrolled over time.
Source: Ha & Schmidhuber (2018), Section 2.2;
Appendix A.2.

Architecture

- LSTM memory: 256 units (Car Racing); 512 (VizDoom).
- Predict each vector entry with five Gaussian components (factored output).
- VizDoom: additional termination head.

Training: make recorded next latent vectors likely (negative log-likelihood).

Design choices

- **Memory:** retain earlier information.
- **Uncertainty:** allow different futures.
- **Latent simulation:** avoid pixel rendering.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ Controller (C)

└ C: A Small Controller

Architecture: join the latent vector and memory, then take weighted sums plus a bias:

$$a_t = W_1 [z_t; h_t] + b_1, \quad a_t = g_{\text{task}}(a_t).$$

 g_{task} applies the task's action bounds and mapping.

Training: freeze V and M; CMA-ES optimises C using rollout return.

cf CMA-ES: Covariance Matrix Adaptation Evolution Strategy.

Design choices

- No hidden layer: keep reward optimisation in a small parameter space.
- Read z_t and h_t : combine current perception with temporal predictive features.
- Reward-only search: sample controllers, rank by return, adapt the search distribution.

C is the policy: it chooses an action from the current latent vector and M's memory. It joins those numbers, takes weighted sums, and adds a bias; this is the affine map in the equation. The task mapping then puts outputs into allowed ranges or discrete choices. In Car Racing, 32 visual values plus 256 memory values produce steering, acceleration, and braking, using 867 parameters. C has no hidden layer, so reward optimisation searches a small set of parameters. CMA-ES means Covariance Matrix Adaptation Evolution Strategy, a gradient-free way to search for good controller weights. It tries controllers sampled from a Gaussian distribution, runs them, and ranks their total rewards. Better controllers guide where to search next. Covariance learns which parameter changes work together; step size controls how large those changes are. V and M stay fixed during this search. The Gaussian is over controller parameters, not future observations. The paper tries 64 candidates per generation, scoring each over 16 seeded rollouts. Next we connect this search to the full training procedure.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ Training Procedure

└ Why Train V, M, and C Separately?

Start with data: collect 10,000 random-policy rollouts for each environment.

Stage	Learning signal	What is learned
1. Train V	Reconstruction + KL	Compact visual representation
2. Train M	Recorded next latent vectors	Action-conditioned predictive memory
3. Optimise C	Rollout return, using CMA-ES	Behaviour from fixed V/M features

Design rationale: learn perception and dynamics from observation data; reserve reward optimisation for the small controller.

Two uses of the same design

- **Car Racing:** optimise C in the actual environment using V/M features.
- **VizDoom:** optimise C inside M's latent simulator, then test in the actual game.

Random interaction first supplies images, actions, and their consequences. V learns image compression, then M learns prediction from the encoded sequences. With V and M fixed, C is optimised for task return. The design gives the large networks reconstruction and prediction targets, while reward optimisation searches only the small controller. This is staged training, not end-to-end reward training of all three networks. For harder tasks, the paper discusses collecting new data as the policy reaches unfamiliar states. Car Racing optimises C in the actual game; VizDoom optimises it inside M's latent simulator and tests it in the actual game. Let us compare those two loops.

Ha & Schmidhuber (2018) - World Models

- Ha & Schmidhuber (2018)

- Training Procedure

- Actual Environment vs. Dream

	Actual game	Dream: M's latent simulator
Current latent vector z_t	V encodes the current image o_t	M supplies a sampled latent vector
Next latent vector z_{t+1}	V encodes the game's next frame	M samples the next latent vector
M's role	Update predictive memory	Memory + latent transitions
C's inputs	z_t plus recurrent memory	z_t plus recurrent memory
V during control	Encoder processes each image	Encoder bypassed; decoder optional for viewing

Why bypass V? The dream already provides latent vectors in V's latent space; there is no rendered frame to encode.

The latent vector is still used. C reads the current z_t and memory. M's sampled z_{t+1} becomes the current latent vector at the next step.

In the actual game, V encodes the current image, and C reads that latent vector plus M's memory to choose an action. The game advances, M updates memory from the latent vector and action, and V encodes the new image at the next decision. In the dream, M instead supplies the next latent vector from its predicted distribution. That sampled latent vector becomes the current observation for the next step. V's encoder is bypassed because there is no image to compress; its decoder is optional for human viewing. The latent vector is still used: z_t represents the current observation, while memory carries temporal context. In VizDoom, memory includes both the LSTM output and cell state. The experiments test whether this setup supports actual-game control and transfer.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ Car Racing

└ Car Racing: Predictive Features for Control



Figure 9: CarRacing-v0.
Source: Ha & Schmidhuber (2018),
Section 9, Table 1.

Features	Score
V only	632 $\pm$ 251
V + M	906 $\pm$ 21

100 random tracks, solve: ≥ 900 .

Task: navigate randomly generated tracks from images. Reward favours visiting track tiles while penalising elapsed time.

Actions: steering, acceleration, braking.

Design

- V: 32-dimensional latent vector; M: 256-unit LSTM.
- C maps $[z_t, h_t]$ to three bounded controls.
- Learn V/M from 10,000 random rollouts; then freeze them.
- CMA-ES optimises C in the **actual game**.

Role of M: predictive features for C, without an explicit planning search at each decision.

The car follows a randomly generated track from images, using steering, acceleration, and braking. Return rewards track coverage and penalises time. Random rollouts train V and M; with them fixed, CMA-ES optimises C in the actual game. C uses M's recurrent features without searching over imagined action sequences at each decision. The reported score rises from 632 with V alone to 906 with V plus M, above the solving threshold over 100 tracks. This shows useful recurrent features, but does not fully separate the effects of memory and predictive training. VizDoom next uses M as C's training environment.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ VizDoom

└ VizDoom: Learning Inside the Dream

Take Cover requires moving sideways to avoid fireballs; return is survival time, up to 2100 steps. Random play in the actual game trains V and M. M predicts both the next latent vector and termination, allowing a complete latent simulator. C reads the current latent vector and both LSTM memory vectors. CMA-ES trains C entirely in the dream, then tests the policy in the actual game. Low temperature gives excellent dream survival but poor transfer; the higher setting reaches a reported actual-game mean of 1092 steps. Added randomness made exploitation harder in this setup, without guaranteeing an accurate model. These results expose both the benefit and risk of learned simulation.



Figure 14. VizDoom: Take Cover.
Source: Ha & Schmidhuber (2018),
Section 4, Table 2.

τ	Dream	Actual
--------	-------	--------

0.10 2086 $\pm$ 140 193 $\pm$ 58

1.15 918 $\pm$ 546 **1092 $\pm$ 556**

Survival steps; actual test: 100

trials. Solve: > 750.

Task: dodge monsters' fireballs and stay alive.

Actions: move left, stay, or move right.

Design

• V: 64-dimensional latent vector; M: 512-unit LSTM plus a termination head.

• C uses $[z_t; h_t; c_t]$: a scalar output maps to three choices.

• Learn V/M from 10,000 recorded random rollouts in the actual game.

• CMA-ES optimizes C **only inside M's latent simulator**, then tests it in the actual game.

Design check: sampling temperature affects model exploitation and transfer.

Ha & Schmidhuber (2018) - World Models

└ Ha & Schmidhuber (2018)

└ Lessons and Limitations

└ Architectural Lessons and Limitations

- **Capacity allocation:** a large V/M representation supports a small reward-optimised controller.
- **V's bottleneck:** reconstruction retains visual information, which may differ from what control needs.
- **M's memory:** a finite LSTM state summarises history; simulated futures advance one step at a time.
- **Simulation errors:** C can exploit M; more stochastic dreams helped transfer but did not eliminate model bias.

V and M learn useful representations before reward optimisation trains the small C. However, V's reconstruction objective can preserve visual detail that differs from what control needs. M compresses history into finite memory and simulates one step at a time. Its errors can affect both the features C reads and the simulated experience used to train C. The temperature results show why a high dream score alone does not establish reliable transfer. The main design separates perception, prediction, and control; the key check is whether their interfaces preserve task-relevant information. Let us summarise those choices.

└ Conclusion

└ Summary

- **V**: a convolutional VAE compresses each image into z_t .
- **M**: an LSTM with a mixture-density head predicts z_{t+1} from z_t , a_t , and memory.
- **C**: a small controller reads the current latent vector and recurrent features to choose actions.
- **Training**: learn V and M from recorded interactions, then optimise C for rollout return.
- **Evidence**: predictive features help Car Racing; latent-simulator training transfers in VizDoom, with sensitivity to model errors.

V compresses images, M predicts latent changes, and C chooses bounded actions from vision and memory. Most representation learning uses recorded observations, while reward optimisation trains the small controller. Car Racing shows predictive features supporting control in the actual game. VizDoom shows dream-trained behaviour transferring to the game, with sensitivity to model errors. We now compare this separation of prediction and control with recent models that use latent planning or joint future-and-action generation.

Ha & Schmidhuber (2018) - World Models

└ Conclusion

└ Comparison with Recent Models

Comparison with Recent Models

Model	Prediction	Action selection	Main design difference
World Models (2018)	VAE latent vectors via MDN-LSTM	Small C, optimised by CMA-ES	Separate V/M/C; task-specific game data
V-JEPA 2-AC (2025)	Future JEPA features	MPC towards an image goal	Web-video pretraining, then robot dynamics
DreamZero (2026)	Future video and action chunks	Joint video-action diffusion policy	Future generation coupled to executable actions
GigaWorld-Policy (2026)	Actions + future video during training	Action output can skip video generation	Causal masking makes future video optional at inference

Connection: 2018 separates prediction from control; recent designs use prediction for latent planning or integrate it with action generation.

Compare what each model predicts and how it chooses actions. The 2018 design learns task-specific V/M features and a separate reward-optimised C. V-JEPA 2-AC starts from video pre-training, adds action-conditioned robot dynamics, and plans towards an image goal. The AC variant takes robot actions; base V-JEPA 2 pretraining does not. DreamZero jointly predicts future video and executable action chunks in one diffusion policy. GigaWorld-Policy trains with action and future-video targets, but its causal mask lets deployment skip video generation. All use future-related information; representation, data scale, and the connection to action selection differ. This is an architectural comparison, not a comparison of game scores with robot success rates.

Thank you. We can return to the architecture, a component's design choices, or the difference between the two training environments. The references are next if we want to check a result.

└ Conclusion

└ References

- K. Crick. *The Nature of Explanation*. Cambridge University Press, 1983.
- J. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 1998.
- R. S. Sutton. Integrated architectures for learning, planning, and reacting (Dyna). *ICML*, 1990.
- R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- G. Yu and J. Schmidhuber. World Models. [arXiv:1803.10122](https://arxiv.org/abs/1803.10122), 2018.
- M. Amodei et al. V. JEPPI. Self-supervised video models enable understanding, prediction and planning. [arXiv:2008.08891](https://arxiv.org/abs/2008.08891), 2020.
- M. Hausman. The CMA Evolution Strategy: A Tutorial. [arXiv:2003.02712](https://arxiv.org/abs/2003.02712), 2020.
- D. Hafner et al. Learning latent dynamics for planning from pixels (PalaNet). *ICML*, 2018. Domain to control (DynaNet). *ICLR*, 2020.
- J. Schrittwieser et al. Mastering GoAt, Go, shogi and shogi by planning with a learned model (MuZero). *Nature*, 2020.
- V. Lefranc. A path towards autonomous machine intelligence. [OpenReview](https://arxiv.org/abs/2003.02712), 2020.
- J. Brown et al. Gato: Generalize interactive environments. *ICML*, 2020.
- J. Brown et al. RT-2: Vision-language-action models. 2023. In J. Kwon et al. *OpenGL*, 2023. K. Black et al. π_2 . 2024.
- Y. Du et al. Learning universal policies via two guided video generators (UniP). *NeurIPS*, 2023.
- M. Nair. A unified latent action world model. [arXiv:2012.13830](https://arxiv.org/abs/2012.13830), 2020.
- S. Nair et al. World action models are zero-shot policies (DynaNet). [arXiv:2003.02712](https://arxiv.org/abs/2003.02712), 2020.
- S. Nair. *OpenGL: Policy*. An efficient action-conditional world action model. [arXiv:2012.13830](https://arxiv.org/abs/2012.13830), 2020.
- S. Zhang, X. Ding, and W. Zhang. From world models to world action models: A concrete tutorial for robotics. [arXiv:2007.00000](https://arxiv.org/abs/2007.00000), 2020.
- J. Li et al. World action models for robot learning and control: A survey. [arXiv:2007.00000](https://arxiv.org/abs/2007.00000), 2020.

The original paper and appendix provide the figures, architecture details, and experiment results.
The other references support the definitions and recent-model comparison.